Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «Криптография и сетевая безопасность»
Тема: «Избыточное кодирование данных в информационных системах. Код Хемминга и итеративные коды»

Выполнил:
Студент 4 курса
Группы АС-66
Гончерёнок К.А.
Проверила:
Хацкевич А.С.

Брест 2026

Цель работы: приобретение практических навыков кодирования/декодирования двоичных данных при использовании кода Хемминга и итеративных кодов.

Вариант 2

Задачи:

1. Закрепить теоретические знания по использованию методов помехоустойчивого кодирования и итеративных кодов для повышения надёжности передачи и хранения в памяти компьютера двоичных данных.
2. Разработать приложение для кодирования/декодирования двоичной информации кодом Хемминга с минимальным кодовым расстоянием 3 и 4; итеративным кодом с различной относительной избыточностью кодовых слов.
3. Результаты выполнения лабораторной работы оформить в виде описания разработанного приложения, методики выполнения экспериментов с использованием приложения и результатов эксперимента.

Hamming.py:

```
"""
Код Хемминга. Поддержка d_min = 3 (классический) и d_min = 4 (расширенный).
"""

import random


def hamming_params(k: int, dmin: int) -> tuple[int, int]:
    """Возвращает (n, r) для заданного k и d_min."""
    r = 1
    while (1 << r) < k + r + 1:
        r += 1
    if dmin == 4:
        return k + r + 1, r + 1
    return k + r, r


def _data_positions(n: int, r: int, dmin: int = 3) -> list[int]:
    """0-индексированные позиции информационных битов."""
    check = set((1 << i) - 1 for i in range(r))
    last = n - 1 if dmin == 4 else n
    return [i for i in range(last) if i not in check]


def _check_positions(n: int, r: int) -> list[int]:
    """0-индексированные позиции проверочных битов (степени 2 – 1)."""
    return [(1 << i) - 1 for i in range(r) if (1 << i) - 1 < n]


def build_check_matrix(k: int, dmin: int = 3) -> tuple[list[list[int]], int, int]:
    """
    Проверочная матрица H (r × n).
    Классический код Хемминга: столбцы — числа 1..n в двоичном виде
    (проверочные позиции 0, 1, 3, 7, ... — там, где cols[j] = 2^i).
    Для d=4 последний столбец — нулевой (общий бит чётности),
         последняя строка — все единицы.
    """
    n, r = hamming_params(k, dmin)
    data_r = r if dmin == 3 else r - 1

    if dmin == 3:
        cols = list(range(1, n + 1))
        H = [[(col >> i) & 1 for col in cols] for i in range(data_r)]
```

```
    else:
        cols = list(range(1, n)) + [0]   # последний — 0 (общий бит)
        H = [[(col >> i) & 1 for col in cols] for i in range(data_r)]
        H.append([1] * n)                # общая проверка на чётность

    return H, n, r


def encode(data: list[int], dmin: int = 3
           ) -> tuple[list[int], list[list[int]], int, int]:
    """Кодирует data длиной k. Возвращает (codeword, H, n, r)."""
    k = len(data)
    H, n, r = build_check_matrix(k, dmin)
    data_r = r if dmin == 3 else r - 1
    data_pos = _data_positions(n, data_r, dmin)
    check_pos = _check_positions(n, data_r)

    cw = [0] * n
    for idx, pos in enumerate(data_pos):
        if idx < k:
            cw[pos] = data[idx]

    # Итеративно правим проверочные биты, пока синдром не станет нулевым
    for _ in range(data_r + 2):
        s = [sum(H[i][j] * cw[j] for j in range(n)) & 1
             for i in range(data_r)]
        if not any(s):
            break
        for i in range(data_r):
            if s[i]:
                cw[check_pos[i]] ^= 1

    # Для d=4: общий бит чётности
    if dmin == 4:
        cw[n - 1] = sum(cw[:n - 1]) & 1

    return cw, H, n, r


def syndrome(cw: list[int], H: list[list[int]]) -> list[int]:
    """S = H · cwᵀ (mod 2)."""
    return [sum(row[i] * cw[i] for i in range(len(cw))) & 1 for row in H]


def decode(received: list[int], H: list[list[int]], dmin: int = 3
           ) -> tuple[list[int], list[int], list[int]]:
    """Декодирует. Возвращает (corrected, syndrome, error_vector)."""
    s = syndrome(received, H)
    n = len(received)
    err = [0] * n

    if not any(s):
        return received[:], s, err

    if dmin == 3:
        # Синдром = номер позиции ошибки (1-индексация)
        s_int = sum(b << i for i, b in enumerate(s))
        if 1 <= s_int <= n:
            err[s_int - 1] = 1
    else:
        H_core = H[:-1]
        s_core = s[:-1]
        s_core_int = sum(b << i for i, b in enumerate(s_core))
        for j in range(n):
            col_int = sum(H_core[i][j] << i for i in range(len(H_core)))
            if col_int == s_core_int:
                err[j] = 1
                break
        if not any(s_core) and s[-1] == 1:
```

```
            err[-1] = 1

    corrected = [received[i] ^ err[i] for i in range(n)]
    return corrected, s, err


def inject_errors(cw: list[int], count: int, rng: random.Random | None = None
                  ) -> tuple[list[int], list[int]]:
    """Вносит count случайных ошибок в кодовое слово."""
    rng = rng or random.Random()
    n = len(cw)
    if count > n:
        raise ValueError(f"count > n ({count} > {n})")
    positions = rng.sample(range(n), count)
    err = [0] * n
    for p in positions:
        err[p] = 1
    return [cw[i] ^ err[i] for i in range(n)], err


def self_test(k: int = 4, dmin: int = 3, trials: int = 1000) -> dict:
    """Для каждой кратности 0..n: сколько обнаружено, сколько исправлено."""
    rng = random.Random(42)
    results = {}
    n_full, _ = hamming_params(k, dmin)
    for i in range(0, n_full + 1):
        detected = fixed = total = 0
        for _ in range(trials):
            data = [rng.randint(0, 1) for _ in range(k)]
            cw, H, n, r = encode(data, dmin)
            recv, _ = inject_errors(cw, i, rng)
            corr, s, _ = decode(recv, H, dmin)
            total += 1
            if i == 0 or any(s):
                detected += 1
            if corr == cw:
                fixed += 1
        results[i] = {
            "total": total,
            "detected": detected,
            "fixed": fixed,
            "detect_ratio": detected / total,
            "fix_ratio": fixed / total,
        }
return results
```

Iterative.py:

```
"""
Итеративный код. Вариант 2: k = 20, матрица 4×5, 2 группы паритетов.
"""

import random


def encode(data: list[int], rows: int = 4, cols: int = 5
           ) -> tuple[list[int], list[list[int]]]:
    """Кодирует data длиной rows*cols в итеративный код."""
    k = rows * cols
    if len(data) != k:
        raise ValueError(f"Нужно {k} бит, получено {len(data)}")
    mat = [[data[r * cols + c] for c in range(cols)] for r in range(rows)]

    h = [sum(mat[r]) & 1 for r in range(rows)]
    v = [sum(mat[r][c] for r in range(rows)) & 1 for c in range(cols)]

    cw = data[:] + h + v
    return cw, mat


def decode(cw: list[int], rows: int = 4, cols: int = 5
```

```
        ) -> tuple[list[int], list[int], str]:
    """
    Декодирует. Возвращает (corrected, error_positions, status).
    status: "ok" | "fixed" | "detected" | "undetected"
    """
    k = rows * cols
    n = k + rows + cols
    if len(cw) != n:
        raise ValueError(f"Длина {len(cw)}, ожидалось {n}")

    data = cw[:k]
    h_recv = cw[k:k + rows]
    v_recv = cw[k + rows:]

    mat = [[data[r * cols + c] for c in range(cols)] for r in range(rows)]
    h_calc = [sum(mat[r]) & 1 for r in range(rows)]
    v_calc = [sum(mat[r][c] for r in range(rows)) & 1 for c in range(cols)]

    h_diff = [h_recv[i] ^ h_calc[i] for i in range(rows)]
    v_diff = [v_recv[j] ^ v_calc[j] for j in range(cols)]

    err = [0] * n

    if not any(h_diff) and not any(v_diff):
        return cw[:], err, "ok"

    bad_rows = [i for i in range(rows) if h_diff[i]]
    bad_cols = [j for j in range(cols) if v_diff[j]]

    if len(bad_rows) == 1 and len(bad_cols) == 1:
        r, c = bad_rows[0], bad_cols[0]
        pos = r * cols + c
        err[pos] = 1
        fixed = [cw[i] ^ err[i] for i in range(n)]
        return fixed, err, "fixed"

    if len(bad_rows) == 1 and not bad_cols:
        pos = k + bad_rows[0]
        err[pos] = 1
        fixed = [cw[i] ^ err[i] for i in range(n)]
        return fixed, err, "fixed"

    if len(bad_cols) == 1 and not bad_rows:
        pos = k + rows + bad_cols[0]
        err[pos] = 1
        fixed = [cw[i] ^ err[i] for i in range(n)]
        return fixed, err, "fixed"

    return cw[:], err, "detected"


def inject_errors(cw: list[int], count: int, rng: random.Random | None = None
                  ) -> tuple[list[int], list[int]]:
    rng = rng or random.Random()
    n = len(cw)
    positions = rng.sample(range(n), count)
    err = [0] * n
    for p in positions:
        err[p] = 1
    return [cw[i] ^ err[i] for i in range(n)], err


def inject_rect_errors(cw: list[int], rows: int = 4, cols: int = 5
                       ) -> tuple[list[int], list[int]]:
    """
    Вносит 4 ошибки в вершины прямоугольника (классический
    необнаруживаемый случай для итеративного кода).
    """
    n = len(cw)
    positions = [0, 1, cols, cols + 1]  # (0,0), (0,1), (1,0), (1,1)
    err = [0] * n
    for p in positions:
        err[p] = 1
    return [cw[i] ^ err[i] for i in range(n)], err
```

```
    def self_test(rows: int = 4, cols: int = 5, trials: int = 1000) -> dict:
        """Анализ корректирующей способности: N2/N1 и N3/N1 для каждой кратности."""
        rng = random.Random(42)
        k = rows * cols
        n = k + rows + cols
        results = {}
        for i in range(0, n + 1):
            detected = fixed = total = 0
            for _ in range(trials):
                data = [rng.randint(0, 1) for _ in range(k)]
                cw, _ = encode(data, rows, cols)
                recv, _ = inject_errors(cw, i, rng)
                corr, _, status = decode(recv, rows, cols)
                total += 1
                if i == 0 or status in ("fixed", "detected"):
                    detected += 1
                if status in ("fixed", "ok") and corr == cw:
                    fixed += 1
            results[i] = {
                "total": total,
                "detected": detected,
                "fixed": fixed,
                "detect_ratio": detected / total,
                "fix_ratio": fixed / total,
            }
        return results
```

Результат программы:

```
Код Хемминга, d_min = 3
============================================================
Текст: 'Hello'
Биты: 0100100001100101011011000110110001101111 (40 бит)
Блок k=4, n=7, r=3

Пример блока 0:
  X_k      = 0100
  X_n      = 1001100
  Y_n      = 1001100
  S        = 000
  E_n      = 0000000
  Y_n'     = 1001100
  Исправлено: True

Блоков: 10
Обнаружено: 10/10 (100.00%)
Исправлено: 10/10 (100.00%)

Статистика кода Хемминга (k=4, d_min=3, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3       78.20%        0.00%
         4       79.20%        0.00%
         5      100.00%        0.00%
         6      100.00%        0.00%
         7        0.00%        0.00%
============================================================
Код Хемминга, d_min = 3
============================================================
Текст: 'Hello'
Биты: 0100100001100101011011000110110001101111 (40 бит)
Блок k=4, n=7, r=3

Пример блока 0:
  X_k      = 0100
  X_n      = 1001100
  Y_n      = 1001110
```

```
  S        = 011
  E_n      = 0000010
  Y_n'     = 1001100
  Исправлено: True

Блоков: 10
Обнаружено: 10/10 (100.00%)
Исправлено: 10/10 (100.00%)

Статистика кода Хемминга (k=4, d_min=3, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3       78.20%        0.00%
         4       79.20%        0.00%
         5      100.00%        0.00%
         6      100.00%        0.00%
         7        0.00%        0.00%
============================================================
Код Хемминга, d_min = 3
============================================================
Текст: 'Hello'
Биты: 0100100001100101011011000110110001101111 (40 бит)
Блок k=4, n=7, r=3

Пример блока 0:
  X_k      = 0100
  X_n      = 1001100
  Y_n      = 0001110
  S        = 111
  E_n      = 0000001
  Y_n'     = 0001111
  Исправлено: False

Блоков: 10
Обнаружено: 10/10 (100.00%)
Исправлено: 0/10 (0.00%)

Статистика кода Хемминга (k=4, d_min=3, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3       78.20%        0.00%
         4       79.20%        0.00%
         5      100.00%        0.00%
         6      100.00%        0.00%
         7        0.00%        0.00%
============================================================
Код Хемминга, d_min = 4
============================================================
Текст: 'Hello'
Биты: 0100100001100101011011000110110001101111 (40 бит)
Блок k=4, n=8, r=4

Пример блока 0:
  X_k      = 0100
  X_n      = 10011001
  Y_n      = 10011001
  S        = 0000
  E_n      = 00000000
  Y_n'     = 10011001
  Исправлено: True

Блоков: 10
Обнаружено: 10/10 (100.00%)
Исправлено: 10/10 (100.00%)
```

```
Статистика кода Хемминга (k=4, d_min=4, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3      100.00%        0.00%
         4       80.70%        0.00%
         5      100.00%        0.00%
         6      100.00%        0.00%
         7      100.00%        0.00%
         8        0.00%        0.00%
============================================================
Код Хемминга, d_min = 4
============================================================
Текст: 'Hello'
Биты: 0100100001100101011011000110110001101111 (40 бит)
Блок k=4, n=8, r=4

Пример блока 0:
  X_k      = 0100
  X_n      = 10011001
  Y_n      = 11011001
  S        = 0101
  E_n      = 01000000
  Y_n'     = 10011001
  Исправлено: True

Блоков: 10
Обнаружено: 10/10 (100.00%)
Исправлено: 10/10 (100.00%)

Статистика кода Хемминга (k=4, d_min=4, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3      100.00%        0.00%
         4       80.70%        0.00%
         5      100.00%        0.00%
         6      100.00%        0.00%
         7      100.00%        0.00%
         8        0.00%        0.00%
============================================================
Код Хемминга, d_min = 4
============================================================
Текст: 'Hello'
Биты: 0100100001100101011011000110110001101111 (40 бит)
Блок k=4, n=8, r=4

Пример блока 0:
  X_k      = 0100
  X_n      = 10011001
  Y_n      = 01011001
  S        = 1100
  E_n      = 00100000
  Y_n'     = 01111001
  Исправлено: False

Блоков: 10
Обнаружено: 10/10 (100.00%)
Исправлено: 0/10 (0.00%)

Статистика кода Хемминга (k=4, d_min=4, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3      100.00%        0.00%
         4       80.70%        0.00%
```

```
         5      100.00%        0.00%
         6      100.00%        0.00%
         7      100.00%        0.00%
         8        0.00%        0.00%
============================================================
Итеративный код, вариант 2: k=20, 4×5, 2 группы паритетов
============================================================
Пример блока 0:
  X_k   = 01001000011001010110
  X_n   = 0100100001100101011001010110 0 (n=29)
  Y_n   = 0100100001100101011001010110 0
  Статус: ok

Блоков: 2
Обнаружено: 2/2 (100.00%)
Исправлено: 2/2 (100.00%)

Статистика итеративного кода (4×5, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3       99.40%        0.00%
         4       99.40%        0.00%
         5       99.90%        0.00%
         6       99.70%        0.00%
         7       99.80%        0.00%
         8       99.70%        0.00%
         9       99.70%        0.00%
        10       99.60%        0.00%
        11      100.00%        0.00%
        12       99.90%        0.00%
        13       99.90%        0.00%
        14       99.90%        0.00%
        15       99.90%        0.00%
        16       99.90%        0.00%
        17       99.80%        0.00%
        18      100.00%        0.00%
        19       99.90%        0.00%
        20       99.50%        0.00%
        21       99.80%        0.00%
        22       99.80%        0.00%
        23       99.60%        0.00%
        24       99.60%        0.00%
        25      100.00%        0.00%
        26      100.00%        0.00%
        27      100.00%        0.00%
        28      100.00%        0.00%
        29      100.00%        0.00%
============================================================
Итеративный код, вариант 2: k=20, 4×5, 2 группы паритетов
============================================================
Пример блока 0:
  X_k   = 01001000011001010110
  X_n   = 0100100001100101011001010110 0 (n=29)
  Y_n   = 0100100001100101011011010110 0
  Статус: fixed

Блоков: 2
Обнаружено: 2/2 (100.00%)
Исправлено: 2/2 (100.00%)

Статистика итеративного кода (4×5, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3       99.40%        0.00%
         4       99.40%        0.00%
```

```
         5       99.90%        0.00%
         6       99.70%        0.00%
         7       99.80%        0.00%
         8       99.70%        0.00%
         9       99.70%        0.00%
        10       99.60%        0.00%
        11      100.00%        0.00%
        12       99.90%        0.00%
        13       99.90%        0.00%
        14       99.90%        0.00%
        15       99.90%        0.00%
        16       99.90%        0.00%
        17       99.80%        0.00%
        18      100.00%        0.00%
        19       99.90%        0.00%
        20       99.50%        0.00%
        21       99.80%        0.00%
        22       99.80%        0.00%
        23       99.60%        0.00%
        24       99.60%        0.00%
        25      100.00%        0.00%
        26      100.00%        0.00%
        27      100.00%        0.00%
        28      100.00%        0.00%
        29      100.00%        0.00%
============================================================
Итеративный код, вариант 2: k=20, 4×5, 2 группы паритетов
============================================================
Пример блока 0:
  X_k   = 01001000011001010110
  X_n   = 0100100001100101011001010110 0 (n=29)
  Y_n   = 0101100001100101011011010110 0
  Статус: fixed

Блоков: 2
Обнаружено: 2/2 (100.00%)
Исправлено: 0/2 (0.00%)

Статистика итеративного кода (4×5, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3       99.40%        0.00%
         4       99.40%        0.00%
         5       99.90%        0.00%
         6       99.70%        0.00%
         7       99.80%        0.00%
         8       99.70%        0.00%
         9       99.70%        0.00%
        10       99.60%        0.00%
        11      100.00%        0.00%
        12       99.90%        0.00%
        13       99.90%        0.00%
        14       99.90%        0.00%
        15       99.90%        0.00%
        16       99.90%        0.00%
        17       99.80%        0.00%
        18      100.00%        0.00%
        19       99.90%        0.00%
        20       99.50%        0.00%
        21       99.80%        0.00%
        22       99.80%        0.00%
        23       99.60%        0.00%
        24       99.60%        0.00%
        25      100.00%        0.00%
        26      100.00%        0.00%
        27      100.00%        0.00%
        28      100.00%        0.00%
```

```
        29      100.00%        0.00%
============================================================
Итеративный код, вариант 2: k=20, 4×5, 2 группы паритетов
============================================================
Пример блока 0:
  X_k   = 01001000011001010110
  X_n   = 0100100001100101011001010110O (n=29)
  Y_n   = 11011000011001010110110101100
  Статус: detected

Блоков: 2
Обнаружено: 2/2 (100.00%)
Исправлено: 0/2 (0.00%)

Статистика итеративного кода (4×5, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3       99.40%        0.00%
         4       99.40%        0.00%
         5       99.90%        0.00%
         6       99.70%        0.00%
         7       99.80%        0.00%
         8       99.70%        0.00%
         9       99.70%        0.00%
        10       99.60%        0.00%
        11      100.00%        0.00%
        12       99.90%        0.00%
        13       99.90%        0.00%
        14       99.90%        0.00%
        15       99.90%        0.00%
        16       99.90%        0.00%
        17       99.80%        0.00%
        18      100.00%        0.00%
        19       99.90%        0.00%
        20       99.50%        0.00%
        21       99.80%        0.00%
        22       99.80%        0.00%
        23       99.60%        0.00%
        24       99.60%        0.00%
        25      100.00%        0.00%
        26      100.00%        0.00%
        27      100.00%        0.00%
        28      100.00%        0.00%
        29      100.00%        0.00%
============================================================
Итеративный код, вариант 2: k=20, 4×5, 2 группы паритетов
============================================================
Пример блока 0:
  X_k   = 01001000011001010110
  X_n   = 0100100001100101011001010110O (n=29)
  Y_n   = 10001110011001010110010101100
  Статус: ok

Блоков: 2
Обнаружено: 0/2 (0.00%)
Исправлено: 0/2 (0.00%)

Статистика итеративного кода (4×5, trials=1000):
 кратность   обнаружено   исправлено
         0      100.00%      100.00%
         1      100.00%      100.00%
         2      100.00%        0.00%
         3       99.40%        0.00%
         4       99.40%        0.00%
         5       99.90%        0.00%
         6       99.70%        0.00%
         7       99.80%        0.00%
```

```
 8        99.70%        0.00%
 9        99.70%        0.00%
10        99.60%        0.00%
11       100.00%        0.00%
12        99.90%        0.00%
13        99.90%        0.00%
14        99.90%        0.00%
15        99.90%        0.00%
16        99.90%        0.00%
17        99.80%        0.00%
18       100.00%        0.00%
19        99.90%        0.00%
20        99.50%        0.00%
21        99.80%        0.00%
22        99.80%        0.00%
23        99.60%        0.00%
24        99.60%        0.00%
25       100.00%        0.00%
26       100.00%        0.00%
27       100.00%        0.00%
28       100.00%        0.00%
29       100.00%        0.00%
```

Вывод: приобрели практические навыки кодирования/декодирования двоичных данных при использовании кода Хемминга и итеративных кодов.